COMP 8920SEF
Introduction to Artificial Intelligence and Machine Learning

Individual Project (50% of Final Grade)

Due Date: 23:59, 26 April 2026 (Sunday)

Aim

This project aims to let students become familiar with implementing machine learning projects. You will be asked to develop a solution to a problem by designing, implementing, and evaluating multiple candidate models, selecting and recommending the best model, documenting the process and solution in a report, and conducting a presentation to demonstrate the workflow.

Classification Problem

The objective of this classification problem is to predict whether a policyholder will file a claim in the next 6 months based on various attributes related to the policyholder, the insured vehicle, and the environment in which the vehicle operates.

- A **dataset** will be provided to you, which has already been separated into the training set (**train.csv**) and test set (**test.csv**).
- The training set contains 8000 rows of data, while the test set contains 1000 rows of data.
- Please use the training set for the training and validation process. Please use the provided test set to demonstrate an unbiased performance of your model.
- There will be **a hidden test set** containing 1000 rows of data **for grading**.
- The dataset includes a target variable: **OUTCOME** (Column S)
    - Value 0 indicates no claim is submitted in the next 6 months.
    - Value 1 indicates the claim is filed in the next 6 months.
- The dataset also includes 18 columns (Columns A to R) of metadata related to the records.

Here is information about the dataset's column information.

Column	Attribute Name	Description
A	**ID**	*A policy ID.*
B	**AGE**	*The age group of the policyholder.*
C	**GENDER**	*The gender of the policyholder.*
D	**RACE**	*The race of the policyholder.*
E	**DRIVING_EXPERIENCE**	*The number of years the policyholder has been driving.*
F	**EDUCATION**	*The highest level of education degree obtained by the policyholder.*
G	**INCOME**	*The income level of the policyholder.*
H	**CREDIT_SCORE**	*The credit score of the policyholder.*
I	**VEHICLE_OWNERSHIP**	*Indicates whether the policyholder owns the vehicle (1 for yes and 0 for no).*
J	**VEHICLE_YEAR**	*The manufacturing year of the insured vehicle.*
K	**MARRIED**	*The marriage status of the policyholder (1 for yes and 0 for no).*
L	**CHILDREN**	*The policyholder has children or not (1 for yes and 0 for no).*
M	**POSTAL_CODE**	*The postal code of the policyholder's residence.*
N	**ANNUAL_MILEAGE**	*The number of miles the policyholder drives annually.*
O	**VEHICLE_TYPE**	*The type of vehicle insured.*
P	**SPEEDING_VIOLATIONS**	*The number of speeding violations the policyholder has had*
Q	**DUIS**	*The number of Driving Under the Influence (DUI) violations the policyholder has had.*
R	**PAST_ACCIDENTS**	*The number of past accidents the policyholder has been involved in.*
S	**OUTCOME**	*The final outcome of the insurance policy.*

Project Requirements

1. **Model Training**
   Train a model that can perform car insurance claiming classification tasks using **Python**. Implement a **training script** (E.g., **train.ipynb**) to perform the learning procedure and obtain a classification model.
   - You can **apply any techniques that perform classification tasks** in your training
     - Logistic Regression
     - K-Nearest-Neighbors
     - Decision Tree
     - Random Forest
     - Naïve Bayes
     - ANN
     - etc.
   - You must implement a **validation strategy** (e.g., train/validation split or k-fold cross-validation) after the training to ensure the model's ability to generalize to new data.
   - A **loss function** needs to be chosen for your measurement of the learning process.

2. **Machine Learning Procedure**
   The basic procedure of machine learning should be implemented, including:
   - Data analysis
   - Data preprocessing
   - Feature engineering
   - Model selection and implementation (**at least three models must be implemented and evaluated**)
   - Training and validation
   - Model optimization

3. **Model Saving**
   You should **save your best-performing model** (**final_model**) for later use.
   - For the model trained by sklearn, store it as **final_model.pkl**.
   - For the model trained by keras, store it as **final_model.keras**.

4. **Model Testing**
   In the last stage, a **Python script (test.ipynb) needs to be implemented,** which is used to apply your model to perform classification.
   - This script should have functionalities that can receive a test.csv and load the final selected best-performing model for prediction.
   - The script should apply the same preprocessing steps used for training.
   - You will also need to save the classification result into a **result.csv** file.

5. **Programming Deliverables** (Under Folder **Code**):
   - Training Script (**train.ipynb**): A Jupyter Notebook containing the entire training process, including data preprocessing, model training, validation, and saving the trained model.
   - Testing Script (**test.ipynb**): A Jupyter Notebook for loading the trained model, making predictions on test data, and saving the results.
   - Best Model File: The final selected best-performing model file (**final_model.xxx**).
   - Prediction File (**result.csv**): The output file containing the classification results for the test data.

By following these requirements, you will develop a robust machine-learning pipeline for car insurance claim classification, ensuring the model is well-trained, validated, and ready for use.

Project Report

- A report of **4-7 pages** addressing the details of your project implementation
- In the written report, you should include:
  - **Title**: “COMP 8920SEF Project Report”
  - **Subtitle**: Your full name and 8-digit student ID.
  - **Introduction**: This section briefly describes the problem of the project and outlines your implementation workflow.
  - **Data Analysis:** This section briefly introduces the insights or visualizations gained from exploring the data, including the distribution of the target variable and a discussion of any characteristics that might affect model performance (e.g., class distribution, missing values, outliers).
  - **Methods**: This section should include a description of the data preparation, methods adopted for handling issues identified from the dataset (if any), description of candidate models implemented, validation strategy, and hyperparameter tuning approach. Basically, what you have done during project implementation should be included in the section.
  - **Results and Discussion**: Present a clear performance comparison of all implemented models using appropriate evaluation metrics. In addition, summarize the insights gained during the project, including a discussion of each model’s strengths and weaknesses, a feature-importance analysis for the top-performing model, and any other relevant observations (for example, error patterns, limitations, and avenues for improvement).
  - **Conclusion**: This section recommends the best and most accepted model, presenting the final evaluation and performance of that model.

Oral Presentation

- You will take a **5-minute oral presentation** to demonstrate your process of solving the problem using machine learning techniques.
- **Demonstration (3 minutes):**
  - Provide an overview of the problem and the dataset.
  - Explain the key steps in your machine learning workflow.
  - Highlight the performance of your final model using relevant evaluation metrics.
  - Showcase the results of your model on the test data, including how predictions are made and saved.
- **Q&A (2 minutes):**
  - Answer questions from the audience regarding your approach, methodology, and results.
- The presentation will be evaluated on the criteria of clear presentation, effective visuals, and Q&A performance.
- Important Notes:
  - DO NOT demonstrate the training process. Focus on presenting the methods and results rather than the detailed training process.

Submission

- You should submit a **zip file** to OLE, which includes:
  - A **project report in pdf format.**
  - A **demonstration slide in PowerPoint format**.
  - All programming deliverables in Folder **Code**.
  - All training and testing data in Folder **Data**.
  - A **README.txt file** includes the commands you used for running your code and the description of your training data.

Grading Referenced Criteria

The grading is mainly based on the performance of the classification model.

- **Pass Grade**
  - **Satisfying the basic requirements**.
  - Obtaining at least **75%** **accuracy in the testing set**.
    - A **public testing** set (**test.csv**) is provided for self-evaluation.
    - A **hidden testing** set (**NOT released**) will be used for final grading.
    - For example:

```
========== Public Test ==========
Accuracy: 0.808203
Precision: 0.678862
Recall: 0.676113
========== Hidden Test ==========
Accuracy: 0.809179
Precision: 0.721774
Recall: 0.667910
```

- **Higher Grades**
  - **Methodology Completeness:** All necessary procedures should be implemented for your project, i.e., data preprocessing, technique selection, training, validation, testing, measurement, etc. Show them in your report.
  - **Model Performance:** The higher performance gained in the hidden test set will result in a higher grade.
  - **Report Quality:** A higher grade will be given to the report quality, including clarity, organization, depth of analysis, and quality of visualizations.
  - **Code Quality:** The well-documented, modular, and reproducible code will receive a higher grade.

Academic Integrity:

- This is an **individual project**. All work submitted must be your own.
- You may discuss concepts and approaches with classmates, but **do not share code**.
- Any use of **external sources must be properly cited** in your report and code comments.
- **Plagiarism will result in failure** of the project and possible further disciplinary action.

Hints:

- Start early and spend some time on exploratory data analysis.
- Integrate your insights gained from data analysis into your model development, not just applying algorithms.
- Choose the appropriate evaluation metrics and consider the trade-off.
- Test your assumptions and validate your approaches.